

Now that you've successfully worked through most of the issues, below is the **final corrected end-to-end Snowflake Retail Analytics Project** with recommendations applied for:

-  GitHub Actions Deployment
 -  Internal Stages
 -  JWT Authentication
 -  No AWS Required
 -  Unsupported Features Removed
 -  Rerunnable Scripts
 -  Snowpark
 -  Analytics Views
 -  Governance Workaround
-

Phase 1 – Environment Setup

01_environment_setup.sql

```
1  USE ROLE ACCOUNTADMIN;
2
3  CREATE WAREHOUSE IF NOT EXISTS RETAIL_WH
4  WAREHOUSE_SIZE='XSMALL'
5  AUTO_SUSPEND=60
6  AUTO_RESUME=TRUE;
7
8  CREATE DATABASE IF NOT EXISTS RETAIL_DB;
9
10 CREATE SCHEMA IF NOT EXISTS RETAIL_DB.RAW;
11 CREATE SCHEMA IF NOT EXISTS RETAIL_DB.STAGE;
12 CREATE SCHEMA IF NOT EXISTS RETAIL_DB.CURATED;
13 CREATE SCHEMA IF NOT EXISTS RETAIL_DB.ANALYTICS;
```

Phase 2 – Security

02_rbac_setup.sql

```
1  CREATE ROLE IF NOT EXISTS DATA_ENGINEER;
2  CREATE ROLE IF NOT EXISTS DATA_ANALYST;
```

```
3 CREATE ROLE IF NOT EXISTS DATA_SCIENTIST;
4
5 GRANT USAGE ON DATABASE RETAIL_DB TO ROLE DATA_ENGINEER;
6 GRANT USAGE ON DATABASE RETAIL_DB TO ROLE DATA_ANALYST;
7
8 GRANT USAGE ON WAREHOUSE RETAIL_WH TO ROLE DATA_ENGINEER;
9 GRANT USAGE ON WAREHOUSE RETAIL_WH TO ROLE DATA_ANALYST;
```

Phase 3 – Ingestion

03_file_formats.sql

```
1 CREATE FILE FORMAT IF NOT EXISTS RETAIL_DB.STAGE.CSV_FORMAT
2 TYPE = CSV
3 SKIP_HEADER = 1
4 FIELD_OPTIONALLY_ENCLOSED_BY='\"';
```

04_stages.sql

```
1 CREATE STAGE IF NOT EXISTS RETAIL_DB.STAGE.CUSTOMERS_STAGE
2 FILE_FORMAT = RETAIL_DB.STAGE.CSV_FORMAT;
3
4 CREATE STAGE IF NOT EXISTS RETAIL_DB.STAGE.PRODUCTS_STAGE
5 FILE_FORMAT = RETAIL_DB.STAGE.CSV_FORMAT;
6
7 CREATE STAGE IF NOT EXISTS RETAIL_DB.STAGE.STORES_STAGE
8 FILE_FORMAT = RETAIL_DB.STAGE.CSV_FORMAT;
9
10 CREATE STAGE IF NOT EXISTS RETAIL_DB.STAGE.SALES_STAGE
11 FILE_FORMAT = RETAIL_DB.STAGE.CSV_FORMAT;
```

05_tables_raw.sql

```
1 CREATE TABLE IF NOT EXISTS RETAIL_DB.RAW.CUSTOMERS(
2 CUSTOMER_ID NUMBER,
3 NAME STRING,
4 EMAIL STRING,
5 CITY STRING
6 );
7
8 CREATE TABLE IF NOT EXISTS RETAIL_DB.RAW.PRODUCTS(
```

```

9  PRODUCT_ID NUMBER,
10 PRODUCT_NAME STRING,
11 CATEGORY STRING,
12 PRICE NUMBER(10,2)
13 );
14
15 CREATE TABLE IF NOT EXISTS RETAIL_DB.RAW.STORES(
16 STORE_ID NUMBER,
17 STORE_NAME STRING,
18 CITY STRING
19 );
20
21 CREATE TABLE IF NOT EXISTS RETAIL_DB.RAW.SALES(
22 SALE_ID NUMBER,
23 CUSTOMER_ID NUMBER,
24 PRODUCT_ID NUMBER,
25 STORE_ID NUMBER,
26 AMOUNT NUMBER(10,2),
27 SALE_DATE DATE
28 );

```

06_copy_commands.sql

```

1  COPY INTO RETAIL_DB.RAW.CUSTOMERS
2  FROM @RETAIL_DB.STAGE.CUSTOMERS_STAGE
3  ON_ERROR=CONTINUE;
4
5  COPY INTO RETAIL_DB.RAW.PRODUCTS
6  FROM @RETAIL_DB.STAGE.PRODUCTS_STAGE
7  ON_ERROR=CONTINUE;
8
9  COPY INTO RETAIL_DB.RAW.STORES
10 FROM @RETAIL_DB.STAGE.STORES_STAGE
11 ON_ERROR=CONTINUE;
12
13 COPY INTO RETAIL_DB.RAW.SALES
14 FROM @RETAIL_DB.STAGE.SALES_STAGE
15 ON_ERROR=CONTINUE;

```

Phase 4 – Data Quality

```
1 SELECT CUSTOMER_ID,COUNT(*)
2 FROM RETAIL_DB.RAW.CUSTOMERS
3 GROUP BY CUSTOMER_ID
4 HAVING COUNT(*) > 1;
5
6 SELECT *
7 FROM RETAIL_DB.RAW.CUSTOMERS
8 WHERE CUSTOMER_ID IS NULL;
```

Phase 5 – Curated Layer

```
1 CREATE OR REPLACE TABLE RETAIL_DB.CURATED.CUSTOMERS AS
2 SELECT DISTINCT
3 CUSTOMER_ID,
4 UPPER(NAME) NAME,
5 LOWER(EMAIL) EMAIL,
6 CITY
7 FROM RETAIL_DB.RAW.CUSTOMERS;
8
9 CREATE OR REPLACE TABLE RETAIL_DB.CURATED.SALES AS
10 SELECT *
11 FROM RETAIL_DB.RAW.SALES;
```

Phase 6 – Dynamic Tables

```
1 CREATE OR REPLACE DYNAMIC TABLE
RETAIL_DB.CURATED.DT_PRODUCT_SALES
2 TARGET_LAG='1 minute'
3 WAREHOUSE=RETAIL_WH
4 AS
5 SELECT
6 PRODUCT_ID,
7 SUM(AMOUNT) TOTAL_SALES
8 FROM RETAIL_DB.RAW.SALES
9 GROUP BY PRODUCT_ID;
```

Phase 7 – Streams

```
1 CREATE STREAM IF NOT EXISTS RETAIL_DB.CURATED.SALES_STREAM
2 ON TABLE RETAIL_DB.RAW.SALES;
```

Phase 8 – Tasks

```
1 CREATE OR REPLACE TASK RETAIL_DB.CURATED.SALES_TASK
2 WAREHOUSE = RETAIL_WH
3 SCHEDULE = '5 MINUTE'
4 AS
5 INSERT INTO RETAIL_DB.CURATED.SALES
6 SELECT *
7 FROM RETAIL_DB.CURATED.SALES_STREAM;
```

Phase 9 – JSON

```
1 CREATE TABLE IF NOT EXISTS RETAIL_DB.RAW.CUSTOMER_JSON
2 (
3 DATA VARIANT
4 );
5
6 INSERT INTO RETAIL_DB.RAW.CUSTOMER_JSON
7 SELECT PARSE_JSON(
8 '{
9   "customer_id":1,
10  "name":"John"
11 }'
12 );
```

Phase 10 – Time Travel

```
1 SELECT *
2 FROM RETAIL_DB.RAW.CUSTOMERS
3 AT(OFFSET=>-60);
```

Phase 11 – Clone

```
1 DROP DATABASE IF EXISTS RETAIL_DB_DEV;  
2  
3 CREATE DATABASE RETAIL_DB_DEV  
4 CLONE RETAIL_DB;
```

Phase 12 – Performance

Search Optimization

```
1 ALTER TABLE RETAIL_DB.RAW.CUSTOMERS  
2 ADD SEARCH OPTIMIZATION;
```

Replace Materialized View

(Materialized Views unsupported in your account)

```
1 CREATE OR REPLACE VIEW RETAIL_DB.ANALYTICS.VW_DAILY_SALES AS  
2 SELECT  
3 SALE_DATE,  
4 SUM(AMOUNT) TOTAL_SALES  
5 FROM RETAIL_DB.RAW.SALES  
6 GROUP BY SALE_DATE;
```

Phase 13 – External Tables

(No AWS)

16_external_tables.sql

```
1 SELECT  
2 'External Tables skipped - No external cloud configured'  
3 AS STATUS;
```

Phase 14 – Snowpark

Use:

```
1 phase14_snowpark/snowpark_analytics.py
```

The corrected version includes:

- JWT Authentication
- Product Revenue
- Customer Revenue
- Store Revenue
- Daily Revenue
- Top Products

and writes results into:

```
1 RETAIL_DB.CURATED.PRODUCT_SALES_SUMMARY
2 RETAIL_DB.CURATED.CUSTOMER_REVENUE_SUMMARY
3 RETAIL_DB.CURATED.STORE_REVENUE_SUMMARY
4 RETAIL_DB.CURATED.DAILY_REVENUE
5 RETAIL_DB.CURATED.TOP_PRODUCTS
```

Phase 15 – Data Sharing

```
1 CREATE SHARE IF NOT EXISTS RETAIL_SHARE;
```

Phase 16 – Cortex AI

```
1 SELECT SNOWFLAKE.CORTEX.SENTIMENT(
2 'Customer service was excellent'
3 );
```

Phase 17 – Monitoring

```
1 SELECT *
2 FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
3 LIMIT 10;
```

Phase 18 – Cost Optimization

```
1 CREATE RESOURCE MONITOR IF NOT EXISTS RETAIL_MONITOR
2 WITH CREDIT_QUOTA=100;
```

Phase 19 – Governance

Masking policies unsupported in your account.

Replace with:

```
1  SELECT
2  'Governance phase skipped - Unsupported Snowflake edition'
3  AS STATUS;
```

Phase 20 – Analytics Layer

```
1  USE DATABASE RETAIL_DB;
2
3  CREATE OR REPLACE VIEW RETAIL_DB.ANALYTICS.VW_DAILY_SALES AS
4  SELECT
5  SALE_DATE,
6  SUM(AMOUNT) AS TOTAL_REVENUE,
7  COUNT(*) AS TOTAL_ORDERS
8  FROM RETAIL_DB.CURATED.SALES
9  GROUP BY SALE_DATE;
10
11 CREATE OR REPLACE VIEW RETAIL_DB.ANALYTICS.VW_TOP_PRODUCTS AS
12 SELECT
13 PRODUCT_ID,
14 SUM(AMOUNT) AS TOTAL_REVENUE
15 FROM RETAIL_DB.CURATED.SALES
16 GROUP BY PRODUCT_ID;
17
18 CREATE OR REPLACE VIEW RETAIL_DB.ANALYTICS.VW_TOP_CUSTOMERS AS
19 SELECT
20 CUSTOMER_ID,
21 SUM(AMOUNT) AS TOTAL_SPENT
22 FROM RETAIL_DB.CURATED.SALES
23 GROUP BY CUSTOMER_ID;
```

Final GitHub Secrets

```
1  SNOWFLAKE_ACCOUNT
```



```
2 SNOWFLAKE_USER
3 SNOWFLAKE_ROLE
4 SNOWFLAKE_WAREHOUSE
5 SNOWFLAKE_DATABASE
6 SNOWFLAKE_PRIVATE_KEY
```

Final Result

After deployment you'll have:

```
1 RAW
2 |— CUSTOMERS
3 |— PRODUCTS
4 |— STORES
5 |— SALES
6
7 CURATED
8 |— CUSTOMERS
9 |— SALES
10 |— PRODUCT_SALES_SUMMARY
11 |— CUSTOMER_REVENUE_SUMMARY
12 |— STORE_REVENUE_SUMMARY
13 |— DAILY_REVENUE
14 |— TOP_PRODUCTS
15
16 ANALYTICS
17 |— VW_DAILY_SALES
18 |— VW_TOP_PRODUCTS
19 |— VW_TOP_CUSTOMERS
```

This version is aligned with the corrections required for your Snowflake edition and GitHub Actions deployment flow.